TUGAS 1

NAMA : Talitha Syifa Al Fath

NIM : 124250173

1. Penjelasan Fungsi

Isi tabel berikut untuk setiap fungsi yang ada dalam program Anda. Jelaskan tujuan fungsi dan cara kerja bagian-bagian pentingnya.

1.1 main( )

Nama Fungsi	`main ( )`
Tujuan Fungsi	Tujuan fungsi `main()` dalam program ini sebagai titik awal masuknya program utama. Sehingga fungsi ini bertanggung jawab dalam memuat data awal dari file cpp ini, untuk menampilkan menu utama serta mengarahkan alur program berdasarkan preferensi yang akan dipilih oleh user.

Potongan Kode	`int main() {`
	`    loadData();`
	
	`    int pilihan;`
	
	`    do {`
	`        cout <<`
	`"\n=========================\n";`
	`        cout << "   SHOPEASE CATALOG     \n";`
	`        cout << "=========================\n";`
	`        cout << "1. Tambah Produk\n";`
	`        cout << "2. Lihat Produk\n";`
	`        cout << "3. Cari Produk\n";`
	`        cout << "4. Keluar\n";`
	`        cout << "=========================\n";`
	`        cout << "Pilih (1-4): ";`
	`        cin >> pilihan;`
	
	`        if (pilihan == 1) {`
	`            tambahProduk();`
	`        } else if (pilihan == 2) {`
	`            tampilkanProduk();`
	`        } else if (pilihan == 3) {`
	`            cariProduk();`
	`        } else if (pilihan == 4) {`

	`saveData();`
	`cout << "\nTerima kasih, sudah menggunakkan ShopEase :)\n";`
	`} else {`
	`cout << "\nPilihan salah! Coba lagi yaa...\n";`
	`}`
	
	`} while (pilihan != 4);`
	
	`return 0;`
	`}`

Penjelasan	Alur kerja fungsi `main()` ini sebagai berikut: 1. `loatData()` ini dipanggil ketika program pertama kali untuk selanjutnya dapat kita baca data produk dari file shopease_db.txt 2. Loop do-while disini digunakan agar menu dapat terus muncul, hingga selanjutnya user memilih 4 opsi (output). 3. Tiap pilihan yang ada user akan diarahkan ke fungsi yang sesuai, seperti pada fungsi `tambahProduk()`, `tampilkanProduk()`, atau `cariProduk()`. 4. Ketika user memilih keluar program. saveData() akan dipanggil untuk nantinya menyimpan perubahan terakhir ke file.

1.2 loadData()

Nama Fungsi	`loadData()`
Tujuan Fungsi	Tujuan fungsi `loadData()` ini bertugas membaca data produk dari file teks shopease_db.txt yang kemudian memuatnya ke dalam array global katalog[]. Maka fungsi ini dipanggil satu kali saat program pertama kali dijalankan.

Potongan Kode	`void loadData() {`
	`FILE *file = fopen("shopease_db.txt", "r");`
	`if (file == NULL) {`
	`cout << "File tidak bisa ditemukan. Mulai lagi dengan data kosong:(\n";`
	`return;`
	`}`
	
	`jumlahProduk = 0;`
	`while (fscanf(file, "%[^|]|%[^|]|%lf|%d|%f\n", katalog[jumlahProduk].name,`

	`katalog[jumlahProduk].category, &katalog[jumlahProduk].price, &katalog[jumlahProduk].stock, &katalog[jumlahProduk].rating) != EOF) {`
	`jumlahProduk++;`
	`}`
	
	`fclose(file);`
	`cout << "Data dimuat. Total produk: " << jumlahProduk << endl;`
	`}`

Penjelasan	Alur kerja fungsi `loadData()` ini sebagai berikut: 1. File dibuka dengan fopen() dalam mode 'r' (read). Jika file tidak ditemukan, fungsi langsung berhenti. 2. fscanf() akan membaca setiap baris, dengan format: nama|kategori|harga|stok|rating. 3. Karakter '|' dipakai sebagai delimiter fscanf menggunakan format %[^|] untuk baca string hingga karakter '|'. 4. Setiap data yang terbaca akan disimpan ke dalam index jumlahProduk pada array katalog[], lalu counter dinaikkan. 5. File nya ditutup dengan fclose() setelah semua data telah selesai dibaca.

1.3 saveData()

Nama Fungsi	`saveData()`
Tujuan Fungsi	Tujuan fungsi `saveData()` ini adalh untuk menyimpan semua data produk yang ada di dalam memori terutama dalam array katalog[] tersebut ke dalam file shopease_db.txt. Selanjutnya akan dipanggil setiap kali ada perubahan data dan saat program keluar.

Potongan Kode	`void saveData() {`
	`FILE *file = fopen("shopease_db.txt", "w");`
	`if (file == NULL) {`
	`cout << "Gagal simpan file!\n";`
	`return;`
	`}`
	
	`for (int i = 0; i < jumlahProduk; i++) {`
	`fprintf(file, "%s|%s|%.0lf|%d|%.1f\n", katalog[i].name, katalog[i].category, katalog[i].price,`

	`katalog[i].stock, katalog[i].rating);`
	`    }`
	
	`    fclose(file);`
	`    cout << "Data tersimpan!\n";`
	`}`

Penjelasan	Alur kerja fungsi `saveData()` ini sebagai berikut: 1. File dibuka dengan mode 'w' (write), ini akan mengganti (nimpa) isi file lama dengan data yang baru. 2. *Loop for* mengiterasi semua produk yang ada di katalog[] dan menuliskan tiap produk ke satu baris. 3. Format penulisan: nama|kategori|harga|stok|rating, ini konsisten dengan format yang dibaca oleh fungsi loadData(). 4. File kemudian ditutup setelah semua data telah berhasil ditulis.

1.4 tambahProduk()

Nama Fungsi	`tambahProduk()`
Tujuan Fungsi	Tujuan fungsi `tambahProduk()` ini akan menangani terkait proses dalam program saya dalam input produk baru dari user dan menyimpannya ke dalam array katalog[]. Setelah data diisi, fungsi langsung memanggil saveData() untuk memastikan data tersimpan ke file.

Potongan Kode	`void tambahProduk() {`
	`    cout << "\n=== TAMBAH PRODUK ===\n";`
	
	`    if (jumlahProduk >= 100) {`
	`        cout << "Katalog penuh!\n";`
	`        return;`
	`    }`
	
	`    cout << "Nama produk: ";`
	`    cin.ignore();`
	`    cin.getline(katalog[jumlahProduk].name,`
	`60);`
	
	`    cout << "Kategori: ";`
	
	`cin.getline(katalog[jumlahProduk].category,`
	`30);`
	
	`    cout << "Harga: Rp ";`

	`cin >> katalog[jumlahProduk].price;`
	
	`cout << "Stok: ";`
	`cin >> katalog[jumlahProduk].stock;`
	
	`cout << "Rating (0-5): ";`
	`cin >> katalog[jumlahProduk].rating;`
	
	`jumlahProduk++;`
	`saveData();`
	
	`cout << "\nProduk Berhasil Ditambahkan!\n";`
	`}`

Penjelasan	Alur kerja fungsi `tambahProduk()` ini sebagai berikut: 1. Pengecekan kapasitas dilakukan pada awal, jika katalog sudah penuh yakni 100 produk maka fungsi berhenti. 2. cin.ignore() ini digunakan untuk membuang karakter newline yang tersisa di buffer sebelum getline(). 3. cin.getline() dipakai untuk nama dan kategori agar bisa menerima input yang mengandung spasi. 4. Data diisi langsung ke dalam slot katalog[jumlahProduk], lalu counter dinaikkan setelah semua field nya terisi. 5. saveData() dipanggil otomatis setelah produk telah ditambahkan, sehingga jadinya data tidak akan hilang jika program tiba-tiba tertutup.

1.5 bubbleSortByName()

Nama Fungsi	`bubbleSortByName()`
Tujuan Fungsi	Tujuan fungsi `bubbleSortByName()` untuk mengurutkan array katalog[] berdasarkan nama produk secara alfabetis (A-Z) menggunakan algoritma *Bubble Sort.*

Potongan Kode	`void bubbleSortByName() {`
	`  for (int i = 0; i < jumlahProduk - 1; i++) {`
	`  for (int j = 0; j < jumlahProduk - i - 1; j++) {`
	`  if (strcmp(katalog[j].name, katalog[j + 1].name) > 0) {`
	`  Produk temp = katalog[j];`
	`  katalog[j] = katalog[j + 1];`

	`katalog[j + 1] = temp;`
	`}`
	`}`
	`}`
	`}`

Penjelasan	Alur kerja fungsi `bubbleSortByName()` ini sebagai berikut: 1. Dua loop bersarang digunakan, loop luar sebagai iterasi 'pass', loop dalam untuk membandingkan elemen bersebelahan. 2. strcmp() digunakan untuk membandingkan dua string secara leksikografis. Jika hasilnya bernilai > 0, maka artinya urutan salah serta kedua elemen ditukar. 3. Pada setiap 'pass', elemen terbesar secara alfabetis akan 'menggelembung' ke posisi akhir, sehingga batas loop dalam dikurangi dengan i. 4. Proses swap menggunakan variabel temp bertipe struct Produk, sehingga seluruh data produk yang bukan hanya nama akan ikut ditukar.

1.6 partition()

Nama Fungsi	`partition()`
Tujuan Fungsi	Tujuan fungsi `partition()` ini adalah untuk sebagai pembantu *Quick Short,* sehingga memilih elemen pivot dan mempartisi array sehingga semua elemen dengan rating lebih besar dari pivot berada di kiri, dan yang lebih kecil berada di kanan.

Potongan Kode	`int partition(Produk arr[], int low, int high) {`
	`float pivot = arr[high].rating;`
	`int i = low - 1;`
	
	`for (int j = low; j < high; j++) {`
	`if (arr[j].rating > pivot) {`
	`i++;`
	`Produk temp = arr[i];`
	`arr[i] = arr[j];`
	`arr[j] = temp;`
	`}`
	`}`
	
	`Produk temp = arr[i + 1];`
	`arr[i + 1] = arr[high];`
	`arr[high] = temp;`

	`return i + 1;` `}`
Penjelasan	Alur kerja fungsi `partition()` ini sebagai berikut: 1. Elemen terakhir (arr[high]) dipilih sebagai pivot. 2. Pada pointer i ini menunjukkan batas elemen yang sudah lebih besar dari pivot. Dimulai dari low-1. 3. Loop j ini mengiterasi seluruh elemen. Jika arr[j].rating > pivot, elemen tersebut ditukarkan ke posisi i+1 (zona kiri). 4. Di akhir loop, pivot ini diletakkan ke posisi yang tepat (i+1), memisahkan elemen lebih besar (kiri) dan lebih kecil (kanan).

1.7 quickSort()

Nama Fungsi	`quickSort()`
Tujuan Fungsi	Tujuan fungsi `quickSort()` adalah mengurutkan array produk berdasarkan ratingnya dari urutan yang tertinggi ke terendah menggunakan algoritma *Quick Sort* secara rekursif.

Potongan Kode	`void quickSort(Produk arr[], int low, int high) {`
	`  if (low < high) {`
	`  int pi = partition(arr, low, high);`
	`  quickSort(arr, low, pi - 1);`
	`  quickSort(arr, pi + 1, high);`
	`  }`
	`}`

Penjelasan	Alur kerja fungsi `quickSort()` ini sebagai berikut: 1. Base case: jika low >= high, artinya sub array hanya memiliki satu elemen atau kosong, sehingga tidak perlu diproses. 2. partition() dipanggil untuk menempatkan satu elemen (pivot) di posisi yang benar dan mengembalikan indeksnya (pi). 3. Setelah partisi, quickSort() dipanggil rekursif untuk subarray kiri (low hingga pi-1) dan subarray kanan (pi+1 hingga high). 4. Pengurutan dilakukan descending (terbesar dulu) karena partition() menempatkan elemen lebih besar ke kiri.

1.8 tampilkanProduk()

Nama Fungsi	`tampilkanProduk()`

Tujuan Fungsi	Tujuan fungsi `tampilkanProduk()` ini akan menampilkan seluruh produk dalam katalog. User bisa memilih apakah ingin melihat produk diurutkan berdasarkan nama *(Bubble Sort)* atau rating *(Quick Sort)* sebelum ditampilkan.
Potongan Kode	

```
void tampilkanProduk() {
    if (jumlahProduk == 0) {
        cout << "\nBelum ada produk.\n";
        return;
    }

    cout << "\n=== PILIH URUTAN ===\n";
    cout << "1. Urutan Nama (A-Z) - Bubble
Sort\n";
    cout << "2. Urutan Rating Tertinggi -
Quick Sort\n";
    cout << "Pilih: ";

    int mode;
    cin >> mode;

    if (mode == 1) {
        bubbleSortByName();
        cout << "\n=== URUTAN NAMA DARI A-Z
===\n";
    } else if (mode == 2) {
        quickSort(katalog, 0, jumlahProduk -
1);
        cout << "\n=== URUTAN DENGAN RATING
TERTINGGI ===\n";
    } else {
        cout << "Pilihan salah!\n";
        return;
    }

    cout <<
"============================================
============\n";
    cout << "No Nama
Kategori  Harga  Stok Rating\n";
    cout <<
"============================================
============\n";

    for (int i = 0; i < jumlahProduk; i++) {
        printf("%-2d %-24s %-10s Rp%.0lf %-4d
```

	`%.1f\n",`
	`  i+1, katalog[i].name, katalog[i].category,`
	`  katalog[i].price, katalog[i].stock, katalog[i].rating);`
	`  }`
	`}`

Penjelasan	Alur kerja fungsi `tampilkanProduk()` ini sebagai berikut:
	1. Pengecekan awal memastikan ada produk sebelum menampilkan menu pilihan urutan.
	2. User memilih mode tampilan:
	  a. mode 1 memanggil bubbleSortByName()
	  b. mode 2 memanggil quickSort().
	6. Setelah pengurutan selesai, loop for mengiterasi seluruh katalog dan mencetak data tiap produk.
	7. printf() dengan format specifier %-Nd digunakan agar kolom tampilan sejajar (left-aligned dengan lebar kolom tetap).

1.9 cariProduk()

Nama Fungsi	`cariProduk()`
Tujuan Fungsi	Tujuan fungsi `cariProduk()` ini akan mencari produk berdasarkan nama yang diinputkan oleh *user* dan menampilkan detail produk yang ditemukan melalui metode pencarian *Linear Search (Sequential Search)*

Potongan Kode	`void cariProduk() {`
	`  if (jumlahProduk == 0) {`
	`  cout << "\nBelum ada produk.\n";`
	`  return;`
	`  }`
	
	`  cout << "\n=== CARI PRODUK ===\n";`
	`  cout << "Nama Produk: ";`
	
	`  char cari[60];`
	`  cin.ignore();`
	`  cin.getline(cari, 60);`
	
	`  for (int i = 0; i < jumlahProduk; i++) {`
	`  if (strcmp(katalog[i].name, cari) == 0) {`

<table>
<tr><td></td><td>

```
            cout << "\nPRODUK DITEMUKAN!\n";
            cout << "Nama     : " <<
katalog[i].name << endl;
            cout << "Kategori : " <<
katalog[i].category << endl;
            cout << "Harga    : Rp " <<
katalog[i].price << endl;
            cout << "Stok     : " <<
katalog[i].stock << endl;
            cout << "Rating   : " <<
katalog[i].rating << endl;
            return;
        }
    }

    cout << "\nProduk tidak ditemukan!\n";
}
```

</td></tr>
<tr><td>Penjelasan</td><td>

Alur kerja fungsi `cariProduk()` ini sebagai berikut:

1. cin.ignore() ini digunakan untuk membersihkan buffer selanjutnya cin.getline() untuk membaca nama produk yang dicari.
2. *Loop for* menelusuri seluruh array katalog[] dari index 0 hingga jumlahProduk-1.
3. strcmp() untuk membandingkan setiap nama produk dengan input user secara case-sensitive dan exact match.
4. Jika produk ditemukan (strcmp() = 0), maka detail produk akan langsung ditampilkan dan fungsi berhenti dengan menggunakan return.
5. Jika seluruh array sudah ditelusuri dan tidak ada yang dirasa cocok, maka pesan 'tidak ditemukan' akan ditampilkan.

</td></tr>
</table>

2. Alasan Pemilihan Algoritma *Searching*

Dalam program yang saya jalankan saya memilih Algoritma *Searching* yaitu *Linear Search (Sequential Search)* sebagai metode pencarian dalam program ShopEase yang saya buat.

Aspek	Penjelasan
Metode	Linear Search (Sequential Search)
Cara Kerja	Array katalog[] ini ditelusuri satu persatu dari index 0, kemudian setiap elemen nantinya akan dibandingkan menggunakan strcmp(). Jika dirasa ditemukan kecocokan maka pencarian akan langsung berhenti serta data produk ditampilkan.
Alasan Dipilih	1. **Data tidak selalu terurut,** sebab array katalog[] ini tidak menjamin urutan tetap alasannya karena pengurutan hanya akan dilakukan saat tampil. Selain itu, Linear Search (Sequential Search) tidak memerlukan data yang udah terurut. 2. **Jumlah data nya kecil**, kapasitas katalog pada program dibatasi 100 produk, sehingga untuk komplektisitas O(n) dari Linear Search (Sequential Search) tidak menjadikan suatu permasalahan performa. 3. **Implementasi sederhana,** Linear Search (Sequential Search) cocok untuk program CRUD (Create Read Update Delete) sederhana dimana tidak adanya kebutuhan pencarian bervolume tinggi. 4. **Searching exact match,** user mencari item berdasarkana nama produk secara persis yang sesuai dengan menggunakan pendekatan iteratif Linear Search (Sequential Search) menggunakan strcmp().
Kompleksitas	**Best Case :** O(1), produk yang dicari ada di index pertama. **Worst Case :** O(n), produk tidak ada atau berada di index terakhir. **Average :** O(n/2) ~ O(n), untuk data secara random (acak).

3. Lampiran

Link Github

https://github.com/Tsyf02/Praktikum-Algoritma-dan-Struktur-Data/blob/d769dd885870b4afa91d209dda1d68a4fd817356/Tugas1_124250173_H.cpp